

RecoTrust — Audit & Cartographie V3.0

Application Streamlit de réconciliation financière

PayMeTrust / Revenu Assurance

Version auditée : **RECOTrust V5.1.8**

Date : 2026-09-24

Classification : Usage interne — Production entreprise

Ce document constitue le rapport d'audit technique, la cartographie d'architecture, l'analyse sécurité / performance / multi-utilisateurs, et le plan d'évolution contrôlée vers une V3.0 professionnelle, **sans réécriture** et **sans modification des règles métier**.

1. Résumé exécutif

RecoTrust est une application Streamlit de réconciliation Payin/Payout entre le système PayMeTrust (PMT) et les fichiers opérateurs (Wave, Orange, MTN, Moov, CinetPay, Bizao, Ifutur...). La base analysée (V5.1.8) est déjà enrichie : authentification PBKDF2, sessions persistantes (sid), matching configurable, upload jusqu'à 4 Go, optimisations mémoire/CSV, module conversion marchand, déploiement systemd/nginx, partenaire Orange SN.

Verdict

- Application **apte à la production** sous réserve des actions sécurité prioritaires listées §8 et §13.
- Les processeurs partenaires et les calculs de réconciliation sont **préservés** (zones protégées).
- **Risque principal** : mot de passe seed hardcodé, fichiers de session livrés dans le ZIP, absence de tests automatisés.

Priorité recommandée : Sécurité → Stabilité → Préservation métier → Performance → Maintenabilité → Nouvelles fonctions.

2. État actuel de l'application

Composant	État V5.1.8	Commentaire
Entrée	main.py	Orchestration UI + flux reco
Auth	auth/* PBKDF2 + rôles USER/ADMIN	OK — seed password à changer en prod
Sessions F5	sid opaque + data/sessions/	OK — ne pas livrer les JSON sessions
Partenaires	13 payin / 12 payout	Lazy import via PROCESSORS
Matching	match_config + selectbox UI	Colonnes utilisateur réellement passées au processeur
Gros volumes	4 Go, sample encoding, cache max_entries	OK pour fichiers ~3 Go
Exports	Excel via utils/excel_export	Préservé
KPI / RA	ra_modules + merchant_conversion	Filtres dates/heures sur KPI & graphes
Tests auto	Absents	À ajouter (non-régression)
Deploy	deploy/systemd + nginx	Prêt instance entreprise / AWS EC2

3. Architecture actuelle

```
Utilisateur
↓ HTTPS (nginx)
Streamlit main.py ← require_auth() / session sid
↓
Upload PMT + Partenaire → validators + data_loader
↓
Config période + colonnes matching (match_config defaults)
↓
get_processor(type, partner) → importlib lazy partenaires/*
↓
process() : prepare_pmt → match → KPI → tabs → export Excel
↓
ra_modules / merchant_conversion (conversion par heure)
```

Modules

Fichier / Package	Responsabilité	Criticité
main.py	UI, upload, config, lancement reco	Critique
partenaires/*	Règles métier par opérateur	Critique métier
partenaires/__init__.py	Registre PROCESSORS lazy	Critique
utils/match_config.py	Clés matching par défaut	Élevée
utils/data_loader.py	Parse CSV/Excel, mémoire	Élevée
utils/pmt_schema.py	Alias colonnes PMT	Élevée
utils/excel_export.py	Rapports Excel	Élevée
utils/merchant_conversion.py	Taux conversion / graphes	Moyenne
auth/*	Auth, sessions, audit, admin	Critique sécu
deploy/*	systemd, nginx, scripts	Production

4. Cartographie des fichiers (synthèse)

Environ 25 processeurs partenaires, 8 modules utils, 7 modules auth, configuration Streamlit, assets, data, logs, deploy.

5. Cartographie des flux

Flux fonctionnel

- Connexion (email + mot de passe) → session sid dans l'URL
- Choix type Payment/Transfer + partenaire
- Upload fichier PMT + fichier partenaire (validation taille/extension)
- Étape config : période + colonnes de matching
- Lancement : processor.process()
- Onglets : Vue globale, Rapport reco, Rapport, Analytics / conversion marchand
- Export Excel final

Flux données

- PMT : CSV/XLSX → prepare_pmt_dataframe → colonnes standard + originales
- Partenaire : parse spécifique processeur → filtre service éventuel → clé matching
- Matching : isin / map sur clés choisies → flags (WAVE, OMCI, OMSN...)
- Résultats en session_state (isolés par session navigateur + sid)

6. Cartographie des règles métier

Les règles de réconciliation (statuts SUCCESS/PENDING/FAILED, frais opérateur, flags de présence bilatérale, écarts, exports) sont **implémentées dans chaque processeur partenaires/***. La V3.0 ne doit pas les factoriser de façon agressive : risque de régression élevé.

Matching configurable : les colonnes sélectionnées dans l'UI sont injectées via match_col_pmt / match_col_partner dans le constructeur du processeur, avec défauts match_config.

7. Cartographie des partenaires

Partenaire	Payin	Payout	Matching PMT défaut	Matching Part. défaut
Bizao	Oui	—	External Transaction Id	Order ID
CinetPay	Oui	Oui	Transaction ID	ID transaction / ID Marchand
Ifutur	Oui	Oui	Transaction ID / ID Op.	external_reference / REFERENCE
Moov CI	Oui	Oui	ID Opérateur	Receipt No.
Moov BF	Oui	Oui	Transaction ID	ThirdPartyTransactionId
MTN CI	Oui	Oui	Transaction ID	Externalid / External id
MTN CM	Oui	Oui	Transaction ID	Externalid / External id
OM BF	Oui	Oui	Transaction ID	FTXN_ID
Orange CI	Oui	Oui	ID Opérateur	Reference
Orange SN	Oui	Oui	Transaction ID	Référence Partenaire
Wave CI/BF/SN	Oui	Oui	Transaction ID	Référence client

Procédure d'ajout : PROCEDURE_NOUVEAU_PARTENAIRE.md (fichier module + PROCESSORS + match_config + py_compile + restart).

8. Audit de sécurité

Points conformes

- Hash mots de passe PBKDF2-HMAC-SHA256, 120k itérations, salt aléatoire
- Pas de secret AWS/API hardcodé détecté dans le code applicatif
- Jeton de session opaque (sid) — pas d'email/rôle dans l'URL métier
- sanitize_query_params — liste blanche page/step/view/sid
- Validation upload : taille max 4 Go, extensions contrôlées (validators)
- Streamlit headless + bind 127.0.0.1 derrière nginx HTTPS (deploy)
- Journal d'audit dédié (auth/audit.py)

Écarts / risques

ID	Constat	Sévérité	Action recommandée
S1	Mot de passe seed hardcodé RecoTrust2026! (users.py)	Élevée	Forcer changement à la 1ère connexion ; retirer du code ; secret e
S2	Fichiers data/sessions/*.json présents dans le ZIP livrable	Moyenne	Exclure du package ; .gitignore strict ; purge TTL
S3	authorized_users.json versionné avec hash	Faible/Moy	Hors repo en prod (RECOTRUST_DATA_DIR)
S4	Pas de rate-limit login explicite	Moyenne	Limiter tentatives (audit + backoff)
S5	showErrorDetails=false (OK) mais logs à surveiller	Faible	Rotation logs logrotate

Isolation multi-utilisateurs

Streamlit isole le session_state par session navigateur. Les résultats de reco et fichiers matérialisés restent dans la session utilisateur. Le sid permet la reprise après F5 sans mélanger les utilisateurs. Attention : le cache @st.cache_data est process-global — les fonctions de chargement doivent rester pures (clé = contenu fichier), ce qui est le cas dans data_loader.

9. Audit performance & gros volumes

- maxUploadSize = 4096 Mo ; validators alignés
- data_loader : détection encoding/delimiter sur échantillon ; moteurs pyarrow→c→python
- prune_stale_file_bytes + release_heavy_session_artifacts + gc logout
- cache DataFrame max_entries réduit pour limiter la RAM multi-utilisateurs
- Processeurs lazy : pas d'import de 25 modules au démarrage

Limites indicatives (instance 4–8 Go RAM) : 2–4 reco simultanées sur fichiers 1–3 Go selon partenaires.

10. Audit multi-utilisateurs

Risques Streamlit classiques maîtrisés pour l'essentiel (pas de globale métier mutable partagée identifiée dans les processeurs). Recommandation : 1 worker Streamlit + reverse proxy ; scaler verticalement avant horizontal (sessions fichier locales).

11. Préparation production / AWS

```
Internet → ALB/HTTPS → nginx (TLS, client_max_body_size 4096m)
    → 127.0.0.1:8501 streamlit (systemd)
    → disque : /var/lib/recotrust/data + /var/log/recotrust
EC2 (ex. m5.xlarge) : 4 vCPU, 16 Go RAM recommandé si fichiers ≥ 2 Go
```

- Utiliser deploy/recotrust.service + nginx.example.conf
- Variables RECOTRUST_DATA_DIR / RECOTRUST_LOG_DIR
- Pas de base de données obligatoire aujourd'hui (fichiers JSON)
- Sauvegarder data/authorized_users.json et logs d'audit

12. Problèmes identifiés

#	Problème	Impact
1	Seed password en clair dans le code	Compromission comptes initiaux
2	Sessions JSON livrées dans le ZIP	Fuite d'état de session
3	Aucun test de non-régression automatisé	Risque lors d'évolutions
4	Hétérogénéité des processeurs (copier-coller)	Maintenance plus coûteuse
5	Dépendance streamlit-extras	Surface de supply-chain

13. Matrice des risques

Modification	Risque	Impact	Proba.	Protection	Rollback
Toucher partenaires/*	Critique	Résultats reco	Haute si modif	Interdit sans validation	ZIP précédent
Changer calculs frais	Critique	Montants	—	Ne pas faire	Git/ZIP
Matching configurable	Faible	Déjà en place	Faible	Défauts match_config	UI reset
Retirer seed password	Moyen	1er login admin	Faible	Bootstrap env	Doc ops
Purge sessions ZIP	Faible	Aucun métier	Nulle	gitignore	N/A

14. Modifications réalisées dans cette livraison audit

- Rapport d'audit & cartographie PDF (ce document)
- Nettoyage livrable : exclusion data/sessions/*.json du package
- Smoke tests registre partenaires (import PROCESSORS + présence modules)
- Aucune modification des règles métier / calculs / processeurs partenaires

15. Modifications volontairement non réalisées

- Refactoring transversal des processeurs (risque métier élevé)
- Migration base de données utilisateurs (non nécessaire court terme)
- Files d'attente / workers async (complexité injustifiée à ce stade)
- Réécriture UI ou changement de framework

16–17. Tests & non-régression

Ajout d'un smoke test structurel (présence modules PROCESSORS). Les tests métier (matched counts, montants) nécessitent des jeux de fichiers représentatifs fournis par l'équipe métier — à brancher en Phase 7 du plan.

18. Architecture cible V3.0 (progressive)

Conserver Streamlit + processeurs partenaires. Renforcer : secrets, tests NR, observabilité, validation colonnes, runbooks AWS. Pas de big-bang.

19. Plan de déploiement

1. Déployer ZIP sur /opt/recotrust (venv + requirements pinés)
2. Configurer RECOTRUST_DATA_DIR / LOG_DIR + droits user recotrust
3. Changer tous les mots de passe seed / comptes initiaux
4. systemctl enable --now recotrust ; valider nginx TLS
5. Test métier : 1 Payment + 1 Transfer sur fichiers connus

20. Plan de rollback

- Conserver ZIP N-1 ; systemctl stop ; restaurer arborescence ; restart
- Restaurer data/authorized_users.json depuis backup

21. Guide utilisateur (essentiel)

- Connexion → type Payment/Transfer → partenaire
- Charger PMT + fichier partenaire → Suivant
- Vérifier/ajuster période et colonnes de matching → Lancer
- Analyser onglets + télécharger le rapport Excel
- F5 : reste connecté, retour écran chargement fichiers (V5.1.2+)

22. Recommandations futures (priorisées)

Priorité	Action	Risque métier
P0	Éliminer seed password du code ; reset comptes prod	Nul si planifié
P0	Ne plus livrer data/sessions dans les ZIP	Nul
P1	Tests NR sur 3 partenaires pivots (Wave CI, Orange CI, MTN)	Faible
P1	Rate-limit login + alerte audit	Nul
P2	Documentation colonnes par partenaire (échantillons)	Nul
P2	logrotate + métriques CPU/RAM process	Nul
P3	Factorisation prudente helpers communs (sans toucher résultats)	Moyen

23. Conclusion

RecoTrust V5.1.8 constitue une base solide de V3.0 opérationnelle : sécurité applicative de bon niveau, matching configurable, multi-partenaires, gros fichiers, déploiement documenté. L'évolution doit rester **incrémentale**, en protégeant strictement partenaires/* et les calculs. Les actions P0 sécurité doivent être appliquées avant exposition large en production.

Document généré automatiquement dans le cadre de la mission Audit / Sécurisation / V3.0 — ne remplace pas la validation métier par les équipes Revenue Assurance.